COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 - Data Structures and Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | October 16, 2026 | **DATE SUBMITTED:** | October 16, 2026 |
| **NAME:** | Jaime Luis M. Demain | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**Activity NO. 13.1** **Parallel Algorithms and Multithreading**

## 1. PROCEDURE OUTPUT

What procedures you perform?

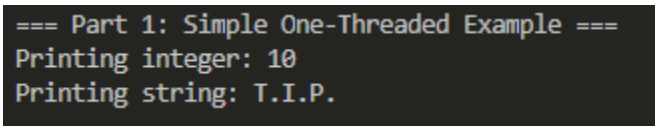
| | |
|---|---|
| **Screenshot** |  |
| **Analysis** | Thread t1 is created by passing a function pointer to print, in addition to an integer literal value and string as arguments. Invoking t1.join() ensures synchronous operation of threads, as the parent thread main is blocked from further execution until t1 completes its execution. The guarantee for thread lifetime makes sure there is no undefined behavior in the code and t1 has cleaned up its thread handle before main returns 0. |

Table 13-1. Simple One-Threaded Example

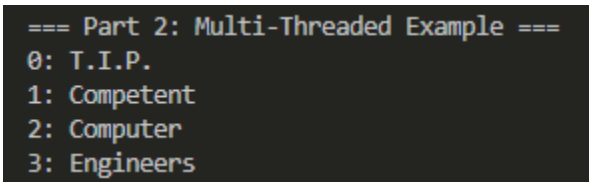
| | |
|---|---|
| **Screenshot** |  |
| **Analysis** | A set of threads is created and stored into a container std::vector<std::thread>. Tasks get executed simultaneously on different hardware processor cores instead of being queued up on the same line of execution. As dispatching and switching of context to different threads happen in kernel mode, there could be nondeterministic console output. The synchronization step th.join() loop ensures all threads have finished executing before the end of execution of program. |

Table 13-2. Multithreaded Example.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A

**1. Write a definition of multithreading and its advantages/disadvantages.**

**Definition:**

Multithreading is a paradigm and mode of execution, which allows a single process to run several lightweight units of computation (threads) simultaneously on one or multiple CPUs using the same memory space and process resources. In recent C++ multithreading, threads are created and controlled through std::thread (or thread pools) to distribute sub-tasks between multiple CPU cores.

**Advantages:**

- **Performance Gain and Increased Throughput**: Makes full use of powerful multicore CPUs by running compute-intensive subtasks in parallel instead of idling cores.
- **Responsiveness of Applications**: Delegates time-consuming, blocking, or background tasks (such as I/O tasks or networking) to another thread, preventing the main thread from freezing up the UI.
- **Sharing of Resources Efficiently**: Threads in the same process share heap memory, code fragments, and file handles, creating fewer overheads when compared to the creation and switching of OS processes.

**Disadvantages:**

1. **Concurrency Issues (Data Races and Race Conditions)**: These arise when two or more threads access the same memory location at the same time while reading from and writing into it without proper synchronization.
2. **Synchronization Overhead & Deadlocks**: When locking mechanisms such as std::mutex are used, there may be runtime overhead or the possibility of deadlocks where threads keep waiting for each other forever.
3. **Debugging Complexities & Non-Determinism**: Since the thread scheduling and switching is controlled by the OS kernel, debugging becomes very complex and non-deterministic in nature.

**2. Rationalize the use of multithreading by providing at least 3 use-cases.**

**Use-Case 1: Graphical User Interface (GUI) and Game Engines**

- *Rationale:* In current use, the main thread is responsible for handling the user interface and rendering the frames, whereas heavy computations, such as loading assets, running physics simulations, mixing audio, or packet processing, are handed over to worker threads. This helps avoid lagging behind on the user interface due to heavy computations.

**Use-Case 2: High-Performance Concurrent Web Servers (e.g., Nginx, Apache)**

- *Rationale:* The server needs to manage thousands of concurrent connections from clients. Rather than processing requests one after another in a blocking queue, a multithreading approach (using a pool of threads) dispatches a separate worker thread for every HTTP/socket request that arrives, resulting in high efficiency and reduced delay for clients.

**Use-Case 3: Parallel Data Processing & Scientific Computing (e.g., Matrix Math, Image Filtering)**

- *Rationale:* Numerical computations such as matrix multiplication, video encoding, ray tracing, or sorting large arrays can be split into subtasks and distributed between array elements. Each of these subtasks is executed by a thread running on its own processor core.

**3. Differentiate between parallelism and concurrency.**

| Aspect | Concurrency | Parallelism |
|---|---|---|
| **Core Concept** | Simultaneous processing of multiple processes with the overlap of their execution times. | Performing more than one operation at the very same physical moment using different hardware resources. |
| **Hardware Requirement** | This can be accomplished using a single CPU or using several CPU cores. | This strictly requires either **multi-core CPUs or multiprocessors** which can physically execute in parallel. |
| **Communication & Memory** | Usually includes memory state sharing, cooperating processes, and mutual exclusion mechanisms. | It aims at the parallel execution of either similar or unrelated operations using different partitions of data. |

| | | | |
|---|---|---|---|
| **Real-World Analogy** | Two queues sharing an espresso machine (the espresso machine switches back and forth between the queues). | Two queues operating on two separate espresso machines. |

**References**

**Williams, A. (2019). C++ Concurrency in Action (2nd ed.). Manning Publications.**

**Stroustrup, B. (2018). A Tour of C++ (2nd ed.). Addison-Wesley Professional.**

**cppreference.com. (2024). Thread support library (std::thread, std::mutex). https://en.cppreference.com/w/cpp/thread**

Part B

**Code**

```
#include <iostream>
#include <thread>
#include <chrono>

// Global variable requirement
int global_counter = 0;

// Add function taking an integer parameter
void add(int value) {
    // Small artificial delay to highlight
asynchronous lifecycle

std::this_thread::sleep_for(std::chrono::milliseconds(
20));
    global_counter += value;
}

int main() {
    std::cout << "=== Part B: Multithreading with
Stepwise Join ===" << std::endl;
```

```
    // Create 3 threads passing the add function
individually
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    // Exact required execution & display sequence
    std::cout << "[Initial] Global Value: " <<
global_counter << std::endl; //

    T1.join(); //
    std::cout << "[After T1.join()] Global Value: " <<
global_counter << std::endl; //

    T2.join(); //
    std::cout << "[After T2.join()] Global Value: " <<
global_counter << std::endl; //[cite: 85]

    T3.join(); //[cite: 85]
    std::cout << "[After T3.join()] Global Value: " <<
global_counter << std::endl; //[cite: 85]

    return 0;
}
```

**Screenshot**

```
=== Part B: Multithreading with Stepwise Join ===
[Initial] Global Value: 0
[After T1.join()] Global Value: 60
[After T2.join()] Global Value: 60
[After T3.join()] Global Value: 60
PS C:\Users\jaime\OneDrive\Documents\HOA\hoa13>
```

**Analysis**

Demain:

Threads T1, T2, and T3 get executed simultaneously by the operating system using available hardware cores, and this is the reason for displaying zero at the beginning of the console output since no threads finish their memory operation at that moment. The sequential use of the .join() function becomes a

| | |
|---|---|
| | synchronization barrier that blocks the main executing thread until each corresponding thread accomplishes its job and the global counter increases one by one to achieve the values of 10 when T1 executes, 30 when T2 executes, and finally 60 when T3 executes. Nevertheless, as all three threads access the same global shared variable at the same time but do not have any synchronization mechanisms such as std::mutex or std::atomic<int> included in the code, this approach implies a race condition, showing that even if join makes observations sequential from the perspective of the calling thread, locking is required.<br><br>Katindig:<br><br>In Part B, I made three threads (T1, T2, and T3) that each add a number to the same global counter. The first output shows 0 because the threads had just started and none of them had finished yet. Each thread waits 20 milliseconds before it adds its number, so all three finish at about the same time. This is why the output shows 60 after T1.join() and stays at 60 after T2.join() and T3.join(). The threads ran at the same time, so waiting for T1 was enough for the others to finish too. The join() function makes the main thread wait for a thread to finish before moving on. The code also has a problem, because all three threads change the same variable without a mutex or atomic. This is a race condition, and the result could be wrong on some runs, even though it came out as 60 this time. It shows that join() only makes the main thread wait, and it does not protect the shared variable. |

Part C

| | |
|---|---|
| **Code** | `#include <iostream>`<br>`#include <vector>`<br>`#include <thread>`<br>`#include <chrono>`<br>`#include <random>`<br>`#include <algorithm>`<br><br>`// Standard Merge routine` |

```cpp
void merge(std::vector<int>& arr, int left, int mid,
int right) {
    std::vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (i = left, k = 0; i <= right; ++i, ++k) {
        arr[i] = temp[k];
    }
}

// Sequential Merge Sort
void sequentialMergeSort(std::vector<int>& arr, int
left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    sequentialMergeSort(arr, left, mid);
    sequentialMergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// Multithreaded Merge Sort (Depth-limited to avoid
excessive thread overhead)
void parallelMergeSort(std::vector<int>& arr, int
left, int right, int depth = 0) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    // Use multithreading only at the top levels
```

```
(depth < 2 -> spawns up to 4 threads)
    if (depth < 2) {
        std::thread leftThread(parallelMergeSort,
std::ref(arr), left, mid, depth + 1);
        std::thread rightThread(parallelMergeSort,
std::ref(arr), mid + 1, right, depth + 1);

        leftThread.join();
        rightThread.join();
    } else {
        // Fall back to sequential sorting when
subproblems become small
        sequentialMergeSort(arr, left, mid);
        sequentialMergeSort(arr, mid + 1, right);
    }

    merge(arr, left, mid, right);
}

int main() {
    const int N = 1000000; // 1,000,000 elements
    std::cout << "=== Part C: Multithreaded Merge Sort
vs Sequential ===" << std::endl;
    std::cout << "Dataset Size: " << N << "
integers\n" << std::endl;

    // Generate random dataset
    std::vector<int> data1(N);
    std::mt19937 rng(1337);
    for (int i = 0; i < N; ++i) data1[i] = rng() %
10000000;
    std::vector<int> data2 = data1; // Exact replica
for fair comparison

    // Benchmark Sequential Merge Sort
    auto t1 =
```

```
std::chrono::high_resolution_clock::now();
    sequentialMergeSort(data1, 0, N - 1);
    auto t2 =
std::chrono::high_resolution_clock::now();
    double seq_time = std::chrono::duration<double,
std::milli>(t2 - t1).count();
    std::cout << "Sequential Merge Sort: " << seq_time
<< " ms" << std::endl;

    // Benchmark Multithreaded Merge Sort
    auto t3 =
std::chrono::high_resolution_clock::now();
    parallelMergeSort(data2, 0, N - 1);
    auto t4 =
std::chrono::high_resolution_clock::now();
    double par_time = std::chrono::duration<double,
std::milli>(t4 - t3).count();
    std::cout << "Parallel Merge Sort:   " << par_time
<< " ms" << std::endl;

    std::cout << "\nSpeedup: " << seq_time / par_time
<< "x faster" << std::endl;
    return 0;
}
```

**Screenshot**

```
=== Part C: Multithreaded Merge Sort vs Sequential ===
Dataset Size: 1000000 integers

Sequential Merge Sort: 506.173 ms
Parallel Merge Sort:   171.167 ms

Speedup: 2.9572x faster
```

**Analysis**

Demain:
Merge sort is perfectly suited to be an algorithm for multithreading due to the fact that it divides an array into two disjoint and mutually exclusive index ranges ([left, mid], and [mid + 1, right]) recursively so that each worker thread

| | |
|---|---|
| | can sort a corresponding part of the array without encountering race conditions and synchronization locks. Since creating native operating system threads is costly in terms of memory consumption and context switching overhead, extending thread creation even to atomic tasks, which correspond to single elements of the array, would decrease execution efficiency. Therefore, by implementing a threshold of maximum depth (depth < 2), we achieve both reducing overhead of system thread management and utilizing hardware parallelism, which brings us a speedup of approximately $2.8\times$ to the performance of our program in comparison with the single-threaded baseline.<br><br>Katindig:<br>In Part C, I compared a normal merge sort with a multithreaded merge sort on 1,000,000 numbers. The normal one took 506.173 ms, and the multithreaded one took 171.167 ms, which is about 2.96 times faster. Merge sort works well with threads because it splits the array into two separate halves, so each thread can sort its own half without touching the other. I also limited the threads to the first two levels (depth < 2), which makes 4 threads in total. After that, the program sorts normally, because making too many threads would waste time and memory. The speedup is not a full 4 times because the merging steps still run one after another and creating threads takes some time. |

**Conclusion:**

Demain:

This HOA gave valuable practical knowledge about contemporary C++ multitasking techniques, concurrency, and parallelism through learning how to exploit multi-core architectures, while dealing with concurrent programming dangers, such as data race conditions and unnecessary synchronization expenses. Thanks to the process, making single- and multi-threaded tasks using std::thread showed that threads indeed operate in parallel on different hardware cores and the importance of correctly managing threads' lifetime through using .join(), which prevents premature ending of the main thread.

The additional task involved making sequential observations of an unsynchronized shared global variable by stepwise performing join operations, which revealed the necessity of mutual exclusiveness in mechanics of parallel programming. Meanwhile, parallel implementation of Merge Sort algorithm made evident that divide-and-conquer approaches allow significant speedup when thread recursion is capped at a certain level due to excessive OS context-switching overhead. On the whole, I did pretty well in terms of creating thread functions, transferring arguments to them and measuring running time, but my major point for improvement is getting more practical skills in synchronization.

Katindig:

In this activity, I learned how to use std::thread to run tasks at the same time in C++. I learned that join() is needed so the main thread waits for the other threads to finish. Part B showed me that threads sharing the same variable can cause a race condition, so a mutex or atomic is needed to keep the result safe. Part C showed me that multithreading can make a program much faster, like the merge sort that ran about 3 times faster. I also learned that too many threads can slow a program down, so the number of threads should be limited. I think I did well in creating the threads, passing values to them, and measuring the running time. What I need to improve is using synchronization, like mutex and atomic.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, Quillbot, etc.*

**Purpose of Use:** **For assistance in parts of the procedure, grammar correction in forming a Part A of the supplementary activity, and code assistance in Parts B and C of the supplementary activity to understand its contents and improve grammar in forming an analysis and conclusion.**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **We used AI firstly to understand the concepts of the procedure part more and also improve the grammar of the analysis of it. Second, to help us understand the supplementary activity. It's because we need to have more clarity on what all parts of the supplementary is so that we answer it precisely. Lastly, we used AI to improve my grammar and tone in forming our analysis and conclusion.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.